



2장. 사이킷런으로 시작하는 머신러닝

3장. 평가

2팀 장서연, 백재은, 고은비

목차

#01 2-01 ~ 2-04 중간

#02 2-04, 2-05

#03 3장



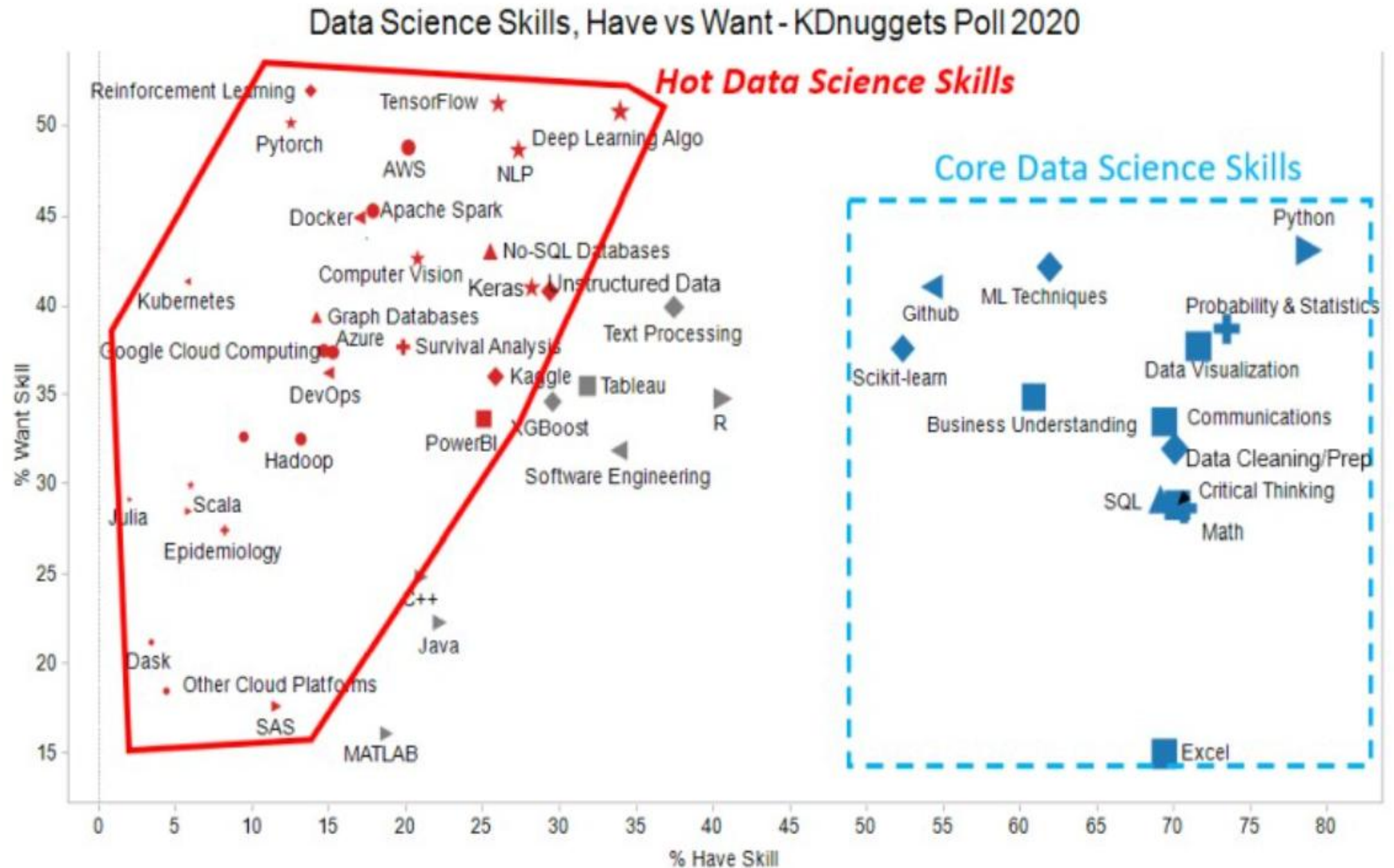
2-01 ~ 2-04 중간



사이킷런 소개

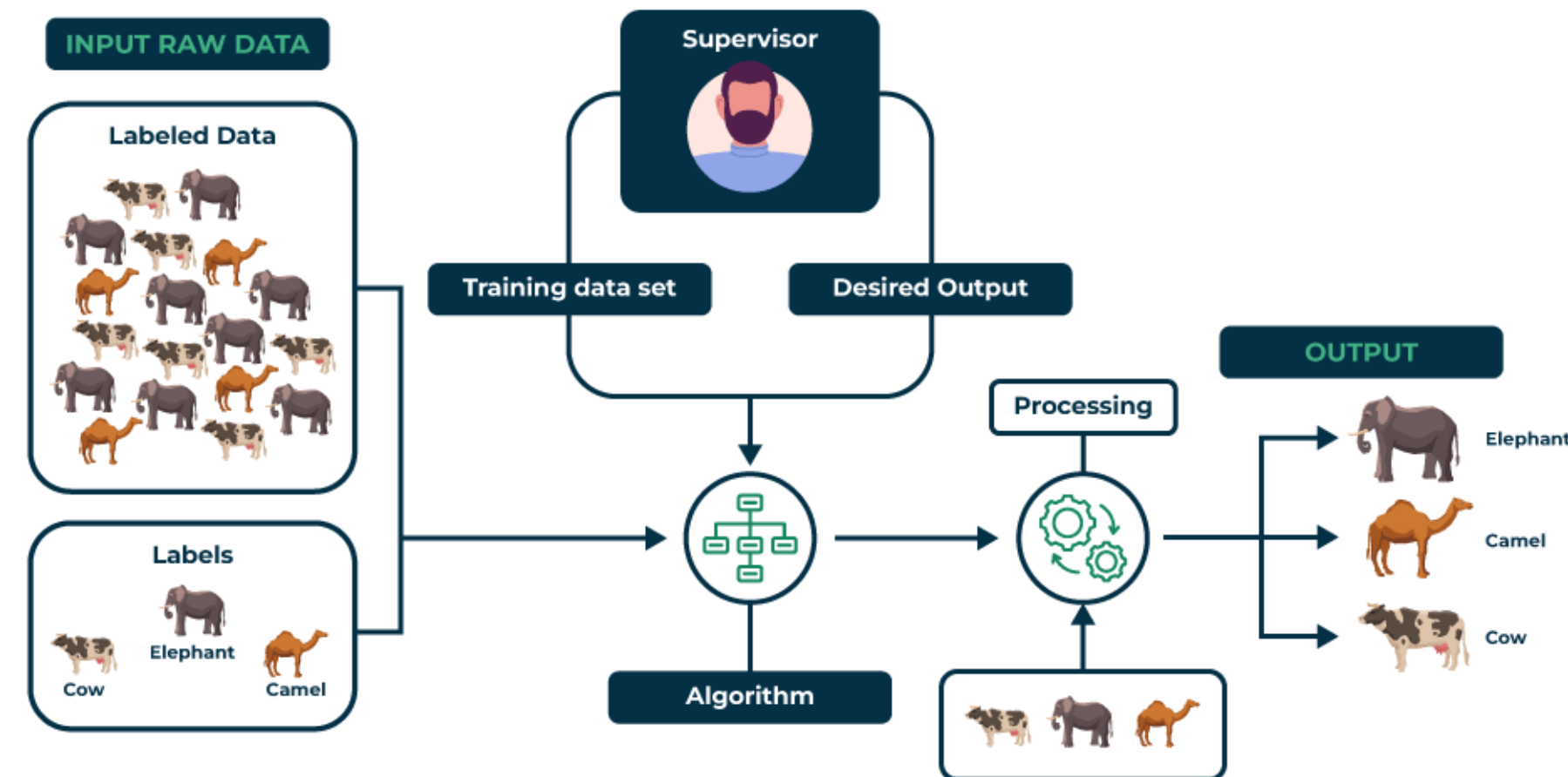


- 가장 파이썬스러운 API
- 머신러닝을 위한 다양한 알고리즘/개발을 위한 프레임워크/API 제공
- 매우 많은 실전 환경에서 사용



지도학습/ 비지도 학습

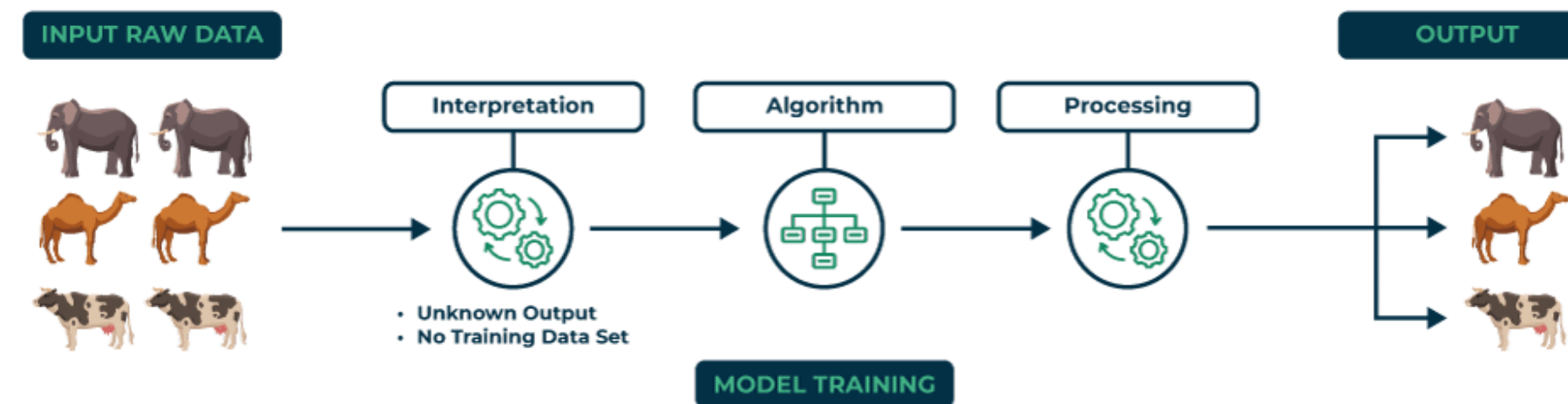
Supervised Learning



지도 학습과 비지도 학습의 가장 큰 차이

→ 정답 / 라벨(Desired Output)의 유무

Unsupervised Learning



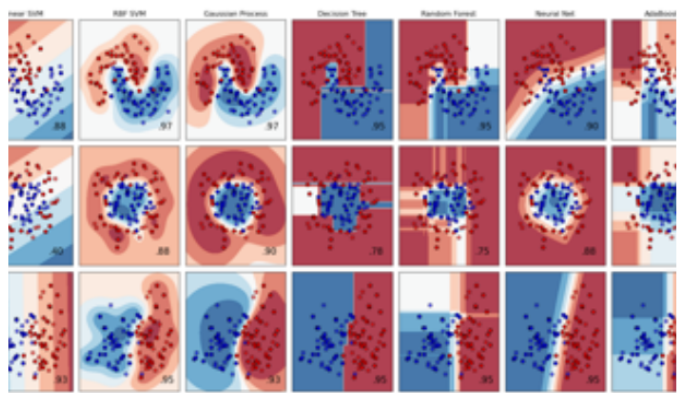
사이킷런 홈페이지

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)



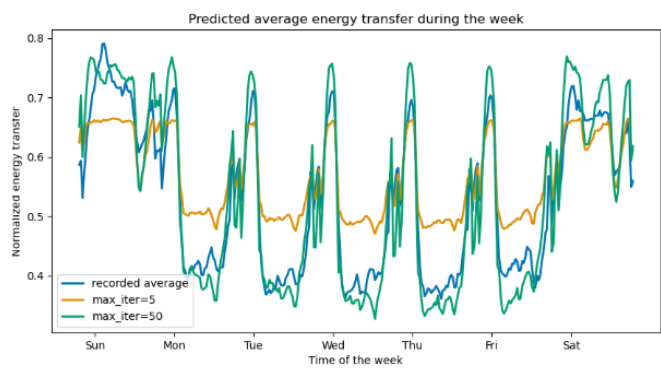
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)



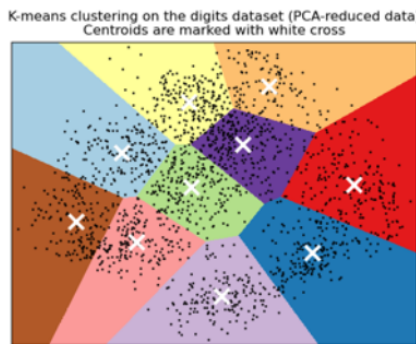
Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.

Algorithms: [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



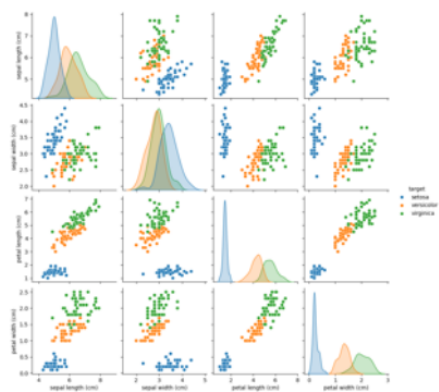
Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, increased efficiency.

Algorithms: [PCA](#), [feature selection](#), [non-negative matrix factorization](#), and [more...](#)



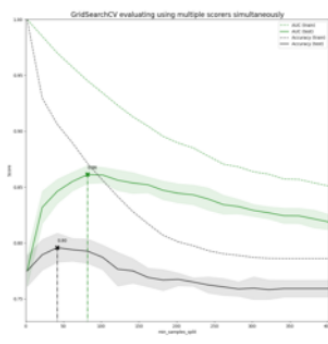
Examples

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning.

Algorithms: [Grid search](#), [cross validation](#), [metrics](#), and [more...](#)



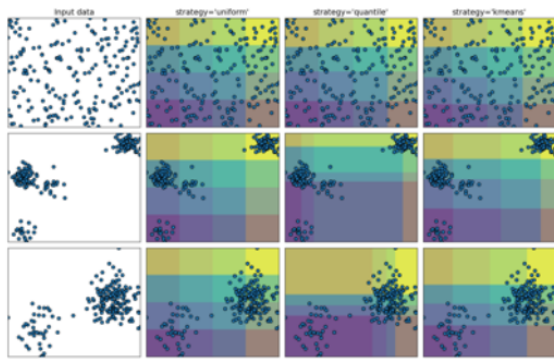
Examples

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: [Preprocessing](#), [feature extraction](#), and [more...](#)



Examples

사이킷런 홈페이지

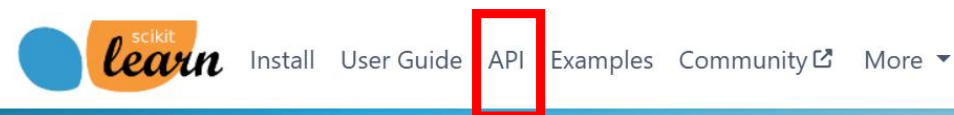
```
import numpy as np
import pandas as pd
import sklearn as sk
import sklearn.모듈
```



```
from sklearn.모듈 import 클래스/함수
```



예시) from sklearn.model_selection import KFold



scikit-learn
Machine Learning in Python

Getting Started

Release Highlights for 1.6

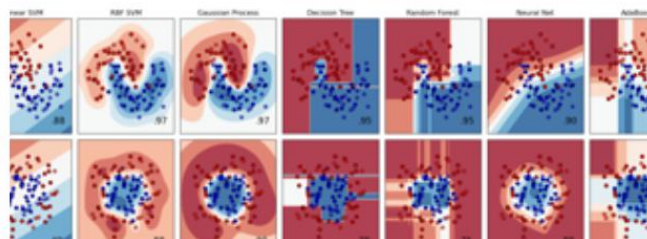
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)

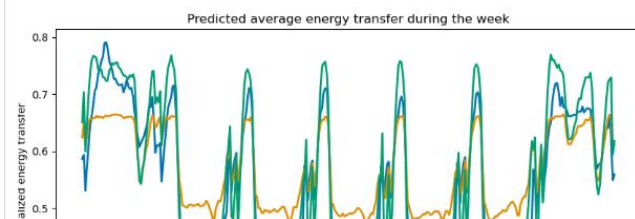


Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)

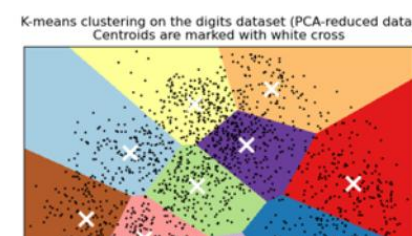


Clustering

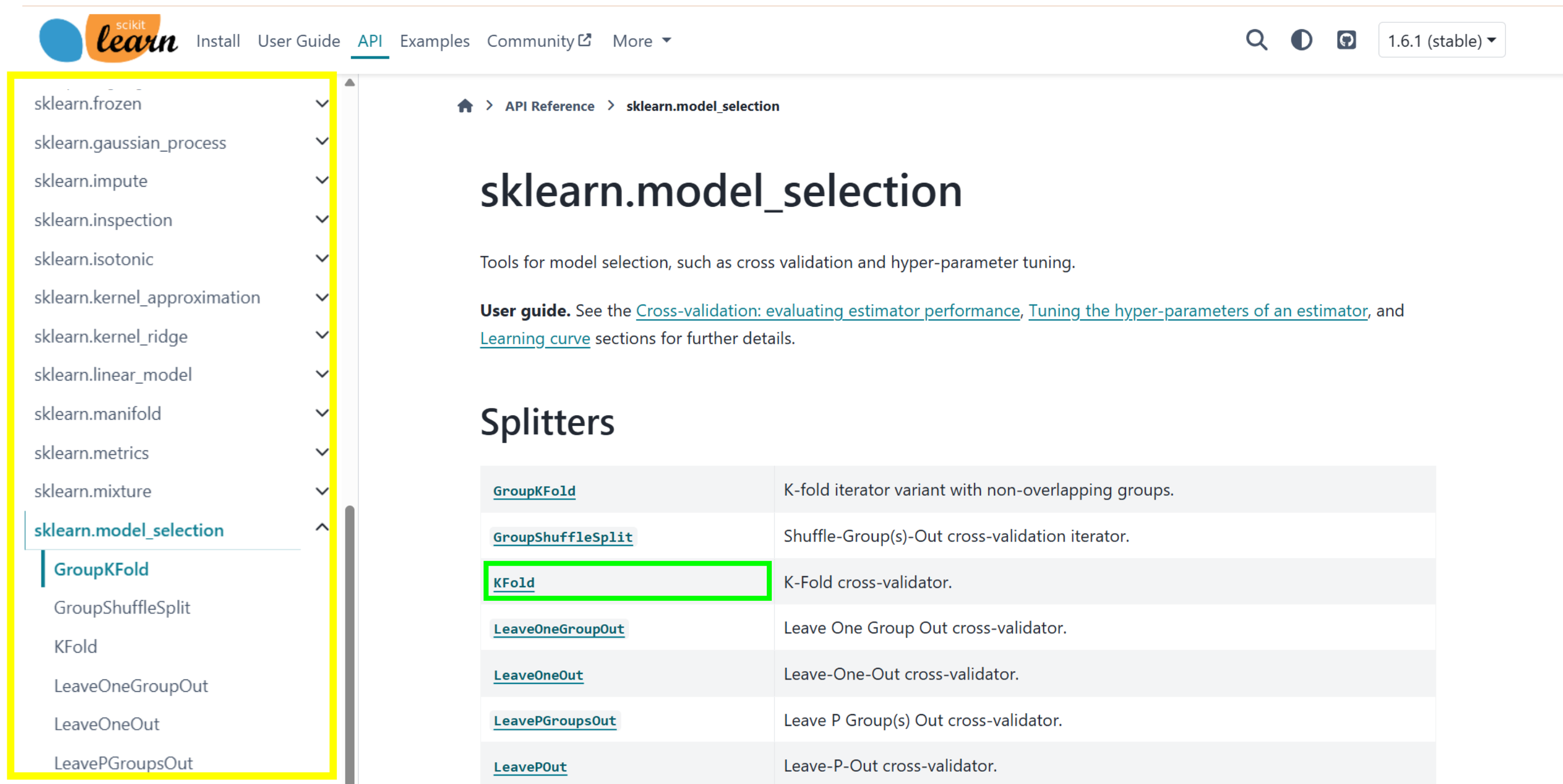
Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.

Algorithms: [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



사이킷런 홈페이지



scikit-learn

Install User Guide API Examples Community More

1.6.1 (stable)

sklearn.frozen
sklearn.gaussian_process
sklearn.impute
sklearn.inspection
sklearn.isotonic
sklearn.kernel_approximation
sklearn.kernel_ridge
sklearn.linear_model
sklearn.manifold
sklearn.metrics
sklearn.mixture
sklearn.model_selection
GroupKFold
GroupShuffleSplit
KFold
LeaveOneGroupOut
LeaveOneOut
LeavePGroupsOut

API Reference > sklearn.model_selection

sklearn.model_selection

Tools for model selection, such as cross validation and hyper-parameter tuning.

User guide. See the [Cross-validation: evaluating estimator performance](#), [Tuning the hyper-parameters of an estimator](#), and [Learning curve](#) sections for further details.

Splitters

GroupKFold	K-fold iterator variant with non-overlapping groups.
GroupShuffleSplit	Shuffle-Group(s)-Out cross-validation iterator.
KFold	K-Fold cross-validator.
LeaveOneGroupOut	Leave One Group Out cross-validator.
LeaveOneOut	Leave-One-Out cross-validator.
LeavePGroupsOut	Leave P Group(s) Out cross-validator.
LeavePOut	Leave-P-Out cross-validator.

from sklearn.모듈 import 클래스/함수 ✓

예시) from sklearn.model_selection import KFold

#사이킷런 홈페이지

예시) from sklearn.model_selection import KFold

KFold

```
class sklearn.model_selection.KFold(n_splits=5, *, shuffle=False, random_state=None)
[source]
```

K-Fold cross-validator.

Provides train/test indices to split data in train/test sets. Split dataset into k consecutive folds (without shuffling by default).

Each fold is then used once as a validation while the k - 1 remaining folds form the training set.

Read more in the [User Guide](#).

For visualisation of cross-validation behaviour and comparison between common scikit-learn split methods refer to [Visualizing cross-validation behavior in scikit-learn](#)

Parameters:

- n_splits** : int, default=5
Number of folds. Must be at least 2.
- Changed in version 0.22:** n_splits default value changed from 3 to 5.
- shuffle** : bool, default=False
Whether to shuffle the data before splitting into batches. Note that the samples within each split will not be shuffled.
- random_state** : int, RandomState instance or None, default=None
When shuffle is True, random_state affects the ordering of the indices, which controls the randomness of each fold. Otherwise, this parameter has no effect. Pass an int for reproducible output across multiple function calls. See [Glossary](#).

Section Navigation

- 1. Supervised learning
- 2. Unsupervised learning
- 3. Model selection and evaluation
 - 3.1. Cross-validation: evaluating estimator performance
 - 3.2. Tuning the hyper-parameters of an estimator
 - 3.3. Tuning the decision threshold for class prediction
 - 3.4. Metrics and scoring: quantifying the quality of predictions
 - 3.5. Validation curves: plotting scores to evaluate models
- 4. Inspection
- 5. Visualizations
- 6. Dataset transformations
- 7. Dataset loading utilities
- 8. Computing with scikit-learn
- 9. Model persistence
- 10. Common pitfalls and recommended practices
- 11. Dispatching
- 12. Choosing the right estimator
- 13. External Resources, Videos and Talks

3.1.2.1.1. K-fold

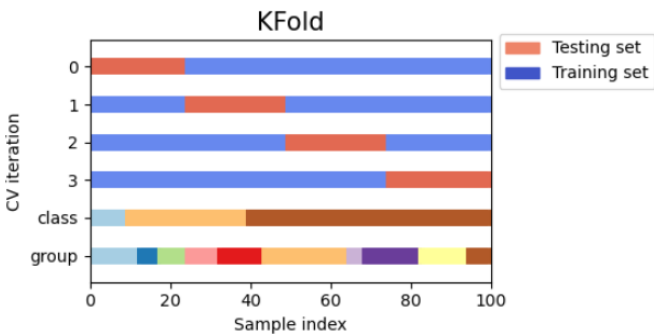
KFold divides all the samples in k groups of samples, called folds (if k = n, this is equivalent to the Leave One Out strategy), of equal sizes (if possible). The prediction function is learned using k - 1 folds, and the fold left out is used for test.

Example of 2-fold cross-validation on a dataset with 4 samples:

```
>>> import numpy as np
>>> from sklearn.model_selection import KFold

>>> X = ["a", "b", "c", "d"]
>>> kf = KFold(n_splits=2)
>>> for train, test in kf.split(X):
...     print("%s %s" % (train, test))
[2 3] [0 1]
[0 1] [2 3]
```

Here is a visualization of the cross-validation behavior. Note that KFold is not affected by classes or groups.



Each fold is constituted by two arrays: the first one is related to the training set, and the second one to the test set. Thus, one can create the training/test sets using numpy indexing:

```
>>> X = np.array([[0., 0.], [1., 1.], [-1., -1.], [2., 2.]])
```

EWCHA
EUROPEAN

iris = load_iris() - Iris 데이터셋 로드

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0

2-2.iris 품종 예측

1. 데이터 세트 분리 – 학습 데이터와 테스트 데이터로 분리

```
X_train, X_test, y_train, y_test = train_test_split(iris_data, iris_label,  
                                                    test_size = 0.2, random_state=11)
```

2. 모델 학습 – 학습 데이터를 기반으로 ML 알고리즘을 이용해 모델 학습

```
# Decision tree 객체 생성  
df_clf = DecisionTreeClassifier(random_state=11)  
  
df_clf.fit(X_train, y_train)
```

3. 예측 수행

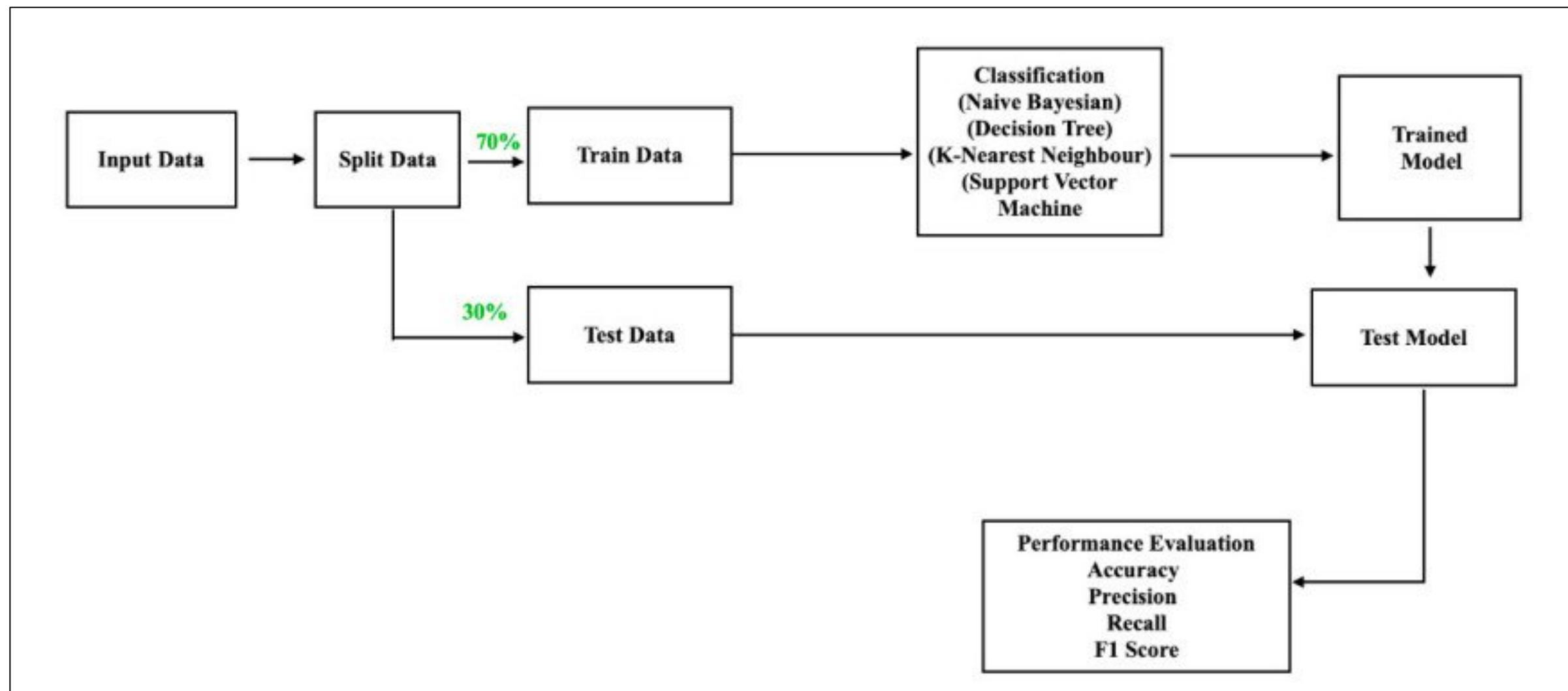
```
pred = df_clf.predict(X_test)
```

4. 성능 평가

```
from sklearn.metrics import accuracy_score  
  
print('예측정확도:{0:.4f}'.format(accuracy_score(y_test, pred)))
```

2-2.iris 품종 예측

Classification 모델의 workflow



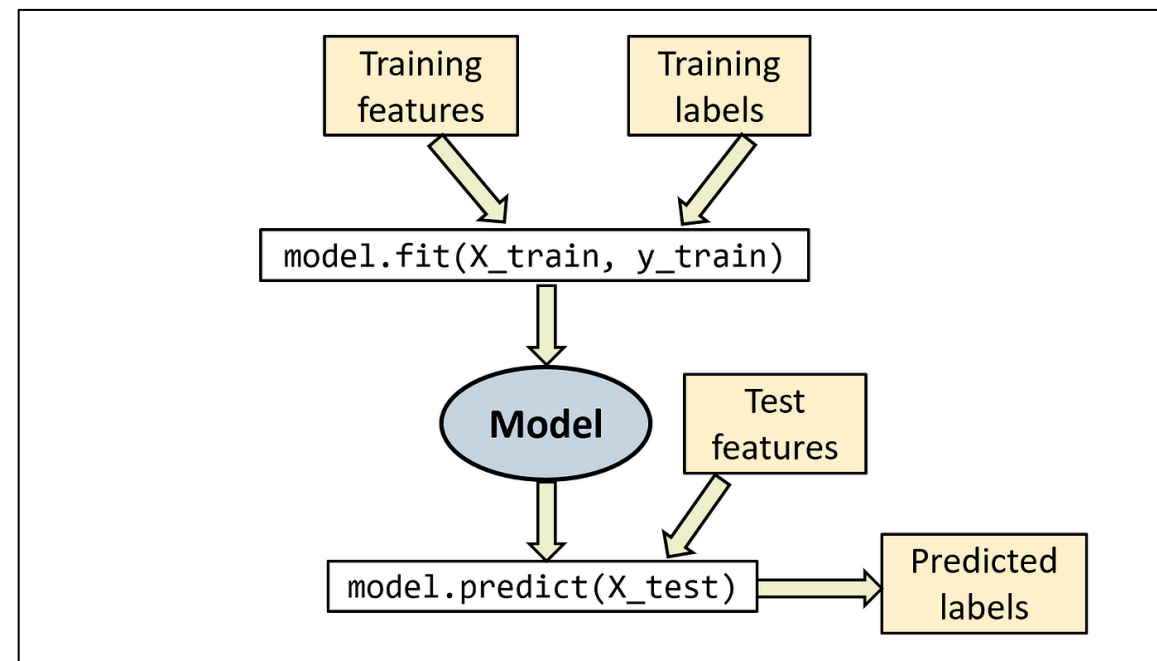
2-3. 사이킷런의 기반 프레임워크

Estimator – 지도학습의 모든 알고리즘을 구현한 클래스
Classifier, Regressor

지도 학습

fit() – ML 모델 학습

predict() – 학습된 모델의 예측



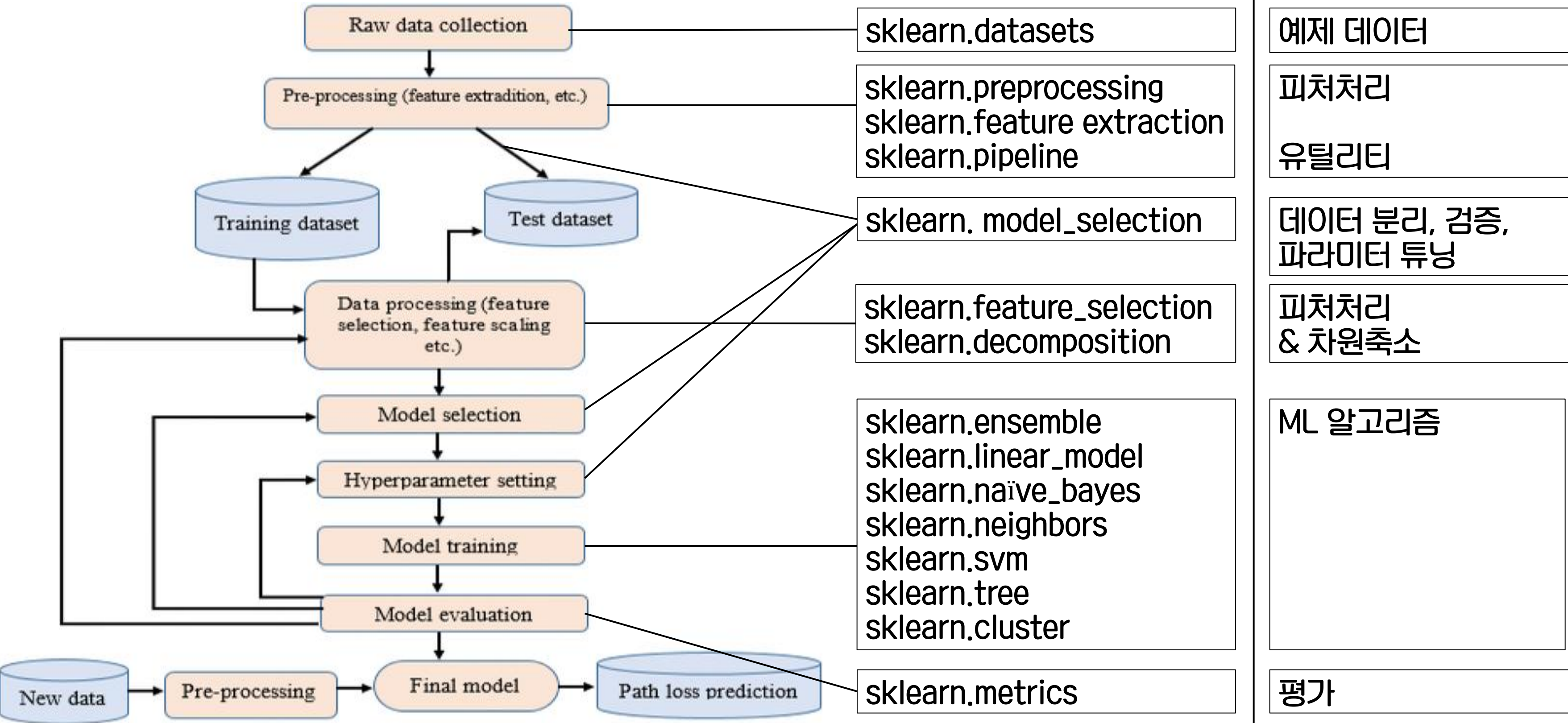
비지도 학습/ 피쳐 추출

fit() –입력 데이터의 형태에 맞춰 데이터를 변환하기 위한 사전 구조를 맞추는 작업

transform() –입력 데이터의 차원 변환, 클러스터링, 피쳐 추출

fit + transform = fit_transform()

사이킷런의 주요 모듈



내장된 예제 데이터 세트

scikit-learn.org > API > sklearn.dataset

Loaders

clear_data_home	Delete all the content of the data home cache.
dump_svmlight_file	Dump the dataset in svmlight / libsvm file format.
fetch_20newsgroups	Load the filenames and data from the 20 newsgroups dataset (classification).
fetch_20newsgroups_vectorized	Load and vectorize the 20 newsgroups dataset (classification).
fetch_california_housing	Load the California housing dataset (regression).
fetch_covtype	Load the covtype dataset (classification).
fetch_file	Fetch a file from the web if not already present in the local folder.
fetch_kddcup99	Load the kddcup99 dataset (classification).
fetch_lfw_pairs	Load the Labeled Faces in the Wild (LFW) pairs dataset (classification).
fetch_lfw_people	Load the Labeled Faces in the Wild (LFW) people dataset (classification).
fetch_olivetti_faces	Load the Olivetti faces data-set from AT&T (classification).
fetch_openml	Fetch dataset from openml by name or dataset id.
fetch_rcv1	Load the RCV1 multilabel dataset (classification).
fetch_species_distributions	Loader for species distribution dataset from Phillips et.
get_data_home	Return the path of the scikit-learn data directory.
load_breast_cancer	Load and return the breast cancer wisconsin dataset (classification).
load_diabetes	Load and return the diabetes dataset (regression).
load_digits	Load and return the digits dataset (classification).
load_files	Load text files with categories as subfolder names.
load_iris	Load and return the iris dataset (classification).
load_linnerud	Load and return the physical exercise Linnerud dataset.
load_sample_image	Load the numpy array of a single sample image.
load_sample_images	Load sample images for image manipulation.
load_svmlight_file	Load datasets in the svmlight / libsvm format into sparse CSR matrix.
load_svmlight_files	Load dataset from multiple files in SVMlight format.
load_wine	Load and return the wine dataset (classification).

Sample generators

make_biclusters	Generate a constant block diagonal structure array for biclustering.
make_blobs	Generate isotropic Gaussian blobs for clustering.
make_checkerboard	Generate an array with block checkerboard structure for biclustering.
make_circles	Make a large circle containing a smaller circle in 2d.
make_classification	Generate a random n-class classification problem.
make_friedman1	Generate the "Friedman #1" regression problem.
make_friedman2	Generate the "Friedman #2" regression problem.
make_friedman3	Generate the "Friedman #3" regression problem.
make_gaussian_quantiles	Generate isotropic Gaussian and label samples by quantile.
make_hastie_10_2	Generate data for binary classification used in Hastie et al. 2009, Example 10.2.
make_low_rank_matrix	Generate a mostly low rank matrix with bell-shaped singular values.
make_moons	Make two interleaving half circles.
make_multilabel_classification	Generate a random multilabel classification problem.
make_regression	Generate a random regression problem.
make_s_curve	Generate an S curve dataset.
make_sparse_coded_signal	Generate a signal as a sparse combination of dictionary elements.
make_sparse_spd_matrix	Generate a sparse symmetric definite positive matrix.
make_sparse_uncorrelated	Generate a random regression problem with sparse uncorrelated design.
make_spd_matrix	Generate a random symmetric, positive-definite matrix.
make_swiss_roll	Generate a swiss roll dataset.

fetch_
인터넷에서 내려받아
scikit_learn_data라는 서브
디렉터리에 저장 후 호출

make_
표본 데이터 생성기

load_iris()

load_iris

```
sklearn.datasets.load_iris(*, return_X_y=False, as_frame=False) \[source\]
```

Load and return the iris dataset (classification).

The iris dataset is a classic and very easy multi-class classification dataset.

Classes	3
Samples per class	50
Samples total	150
Dimensionality	4
Features	real, positive

Classes(라벨) – 0,1,2

Dimensionality – features의 개수

Features – real(실수), positive(양수)

load_iris()

반환 타입 – Bunch Class(딕셔너리형)

Key 값 – 데이터셋.key값/ 데이터셋[key값]

Key	Value(반환값)
.data	2차원 ndarray (150,4) X 값
.target	1차원 ndarray / 판다스 series (150,) 레이블/Y값
.feature_names	피쳐(컬럼)명 리스트
.target_names	레이블명 리스트
.DESCR	str, 데이터셋의 설명
.filename	str, 데이터셋의 경로

feature_names

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
5.1	3.5	1.4	0.2
4.9	3.0	1.4	0.2
.....			
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.2

target_names

setosa, versicolor, virginica
(0 , 1 , 2)

0
1
.....
2
0

data

target

2-4. Model Selection 모듈 소개

train_test_split() – 학습/테스트 데이터셋 분리

- 데이터 셋을 분리하지 않았을 때

```
iris= load_iris()  
dt_clf=DecisionTreeClassifier()  
train_data = iris.data  
train_label = iris.target  
dt_clf.fit(train_data,train_label)  
  
pred = dt_clf.predict(train_data)  
print('예측 정확도:',accuracy_score(train_label,pred))
```

예측 정확도: 1.0

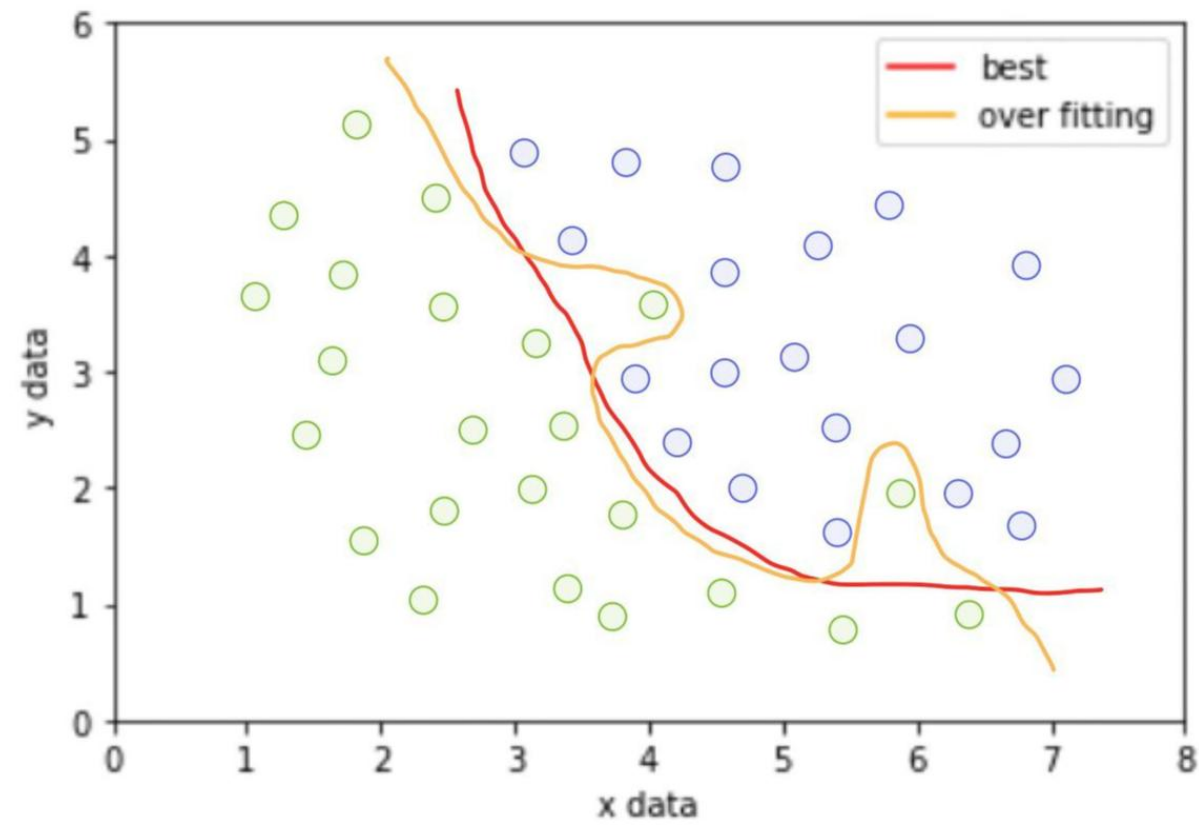
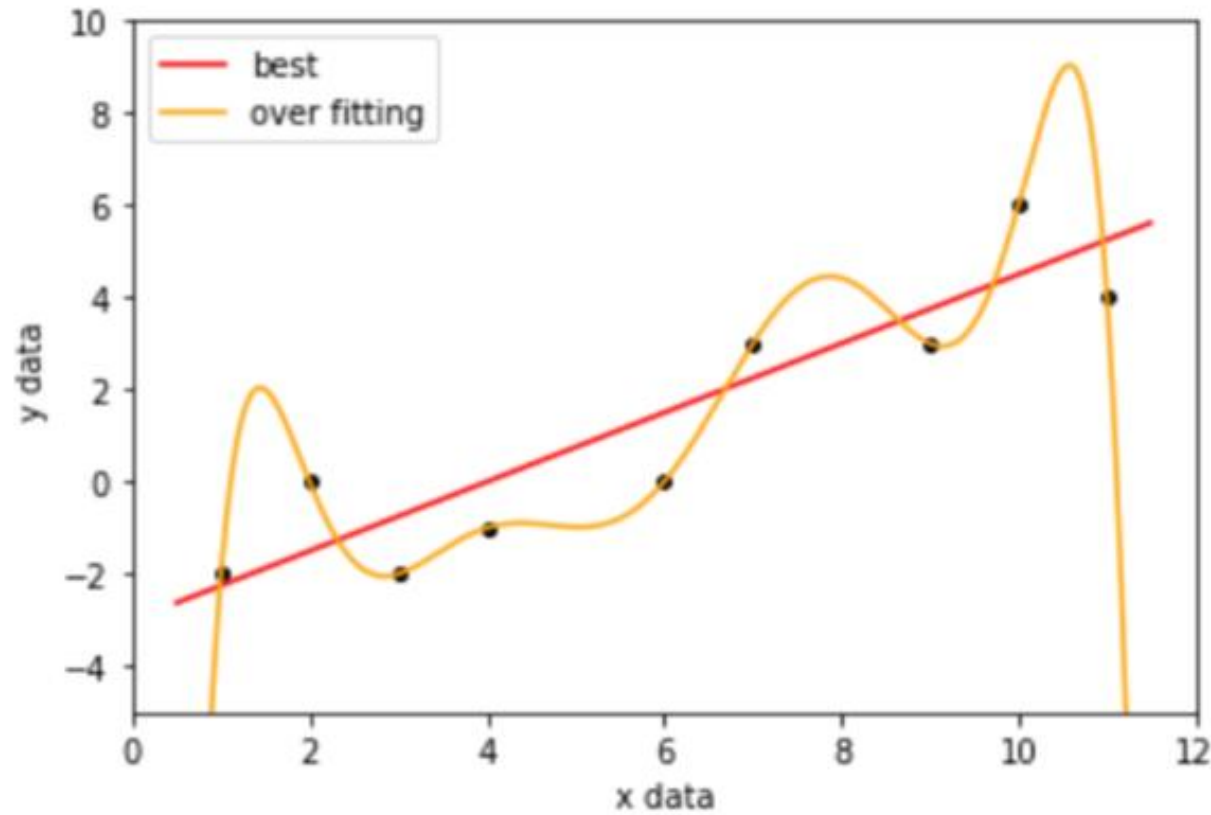
2-4. Model Selection 모듈 소개

```
train_test_split( X(dataset.data), y(dataset.target),
                  test_size = ,train_size = ,
                  random_state = , shuffle = )
```

test_size	(테스트 데이터셋)/(전체 데이터셋) 디폴트값: 0.25 통상적으로 사용
train_size	(학습용 데이터셋)/(전체 데이터셋) 잘 사용하지 않음
random_state	호출할 때마다 학습/테스트용 데이터세트를 생성하기 위해 주어지는 난수 값 / 난수 생성 시드값
shuffle	데이터를 분리하기 전에 데이터를 미리 섞을지 결정 디폴트값 : True

교차 검증

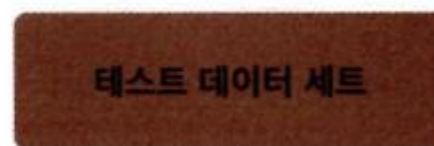
과적합(overfitting)



학습 데이터를 다시 분할하여 학습 데이터와 학습된 모델의 성능을 일차 평가하는 검증 데이터로 나눔

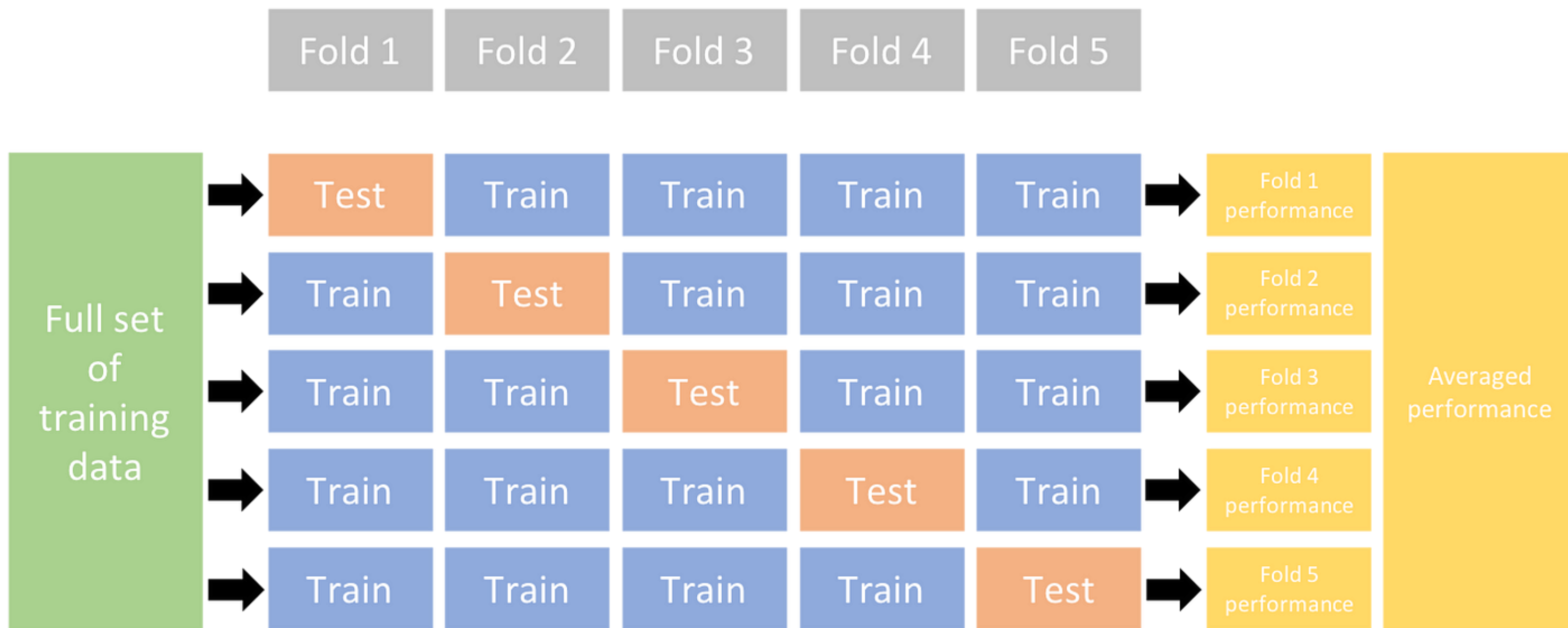


모든 학습/검증 과정이 완료된 후 최종적으로 성능을 평가하기 위한 데이터 세트



K 폴드 교차검증

```
class sklearn.model_selection.KFold(n_splits=5, *, shuffle=False, random_state=None)
```



K 폴드 교차검증

Kfold객체.split(데이터셋(X값))

- X값(features)의 학습데이터셋 인덱스와 검증데이터셋 인덱스를 각 폴드마다 생성함

```
n_iter=0
for train_index, test_index in kfold.split(features):
    X_train,X_test = features[train_index],features[test_index]
    y_train,y_test = label[train_index],label[test_index]
```

np.round(__, 정수)

- Numpy의 반올림함수 / 정수가 양수: 소수점자리, 음수: 정수자리

```
dt_clf.fit(X_train,y_train)
pred=dt_clf.predict(X_test)
n_iter += 1
accuracy = np.round(accuracy_score(y_test,pred),4)
train_size = X_train.shape[0]
test_size = X_test.shape[0]
print('\n#{0} 교차검증 정확도:{1}, 학습 데이터 크기: {2}, 검증데이터 크기: {3}'
      .format(n_iter, accuracy, train_size, test_size))
```

```
print('#{0} 검증 세트 인덱스:{1}'.format(n_iter,test_index))
cv_accuracy.append(accuracy)
```

```
print('\n#평균검증정확도:',np.mean(cv_accuracy))
```

#1 교차검증 정확도:1.0, 학습 데이터 크기: 120, 검증데이터 크기: 30

#1 검증 세트 인덱스:[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29]

#2 교차검증 정확도:0.9667, 학습 데이터 크기: 120, 검증데이터 크기: 30

#2 검증 세트 인덱스:[30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53
54 55 56 57 58 59]

#3 교차검증 정확도:0.8667, 학습 데이터 크기: 120, 검증데이터 크기: 30

#3 검증 세트 인덱스:[60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83
84 85 86 87 88 89]

#4 교차검증 정확도:0.9333, 학습 데이터 크기: 120, 검증데이터 크기: 30

#4 검증 세트 인덱스:[90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107
108 109 110 111 112 113 114 115 116 117 118 119]

#5 교차검증 정확도:0.7333, 학습 데이터 크기: 120, 검증데이터 크기: 30

#5 검증 세트 인덱스:[120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137
138 139 140 141 142 143 144 145 146 147 148 149]

#평균검증정확도: 0.9

1) K값 설정

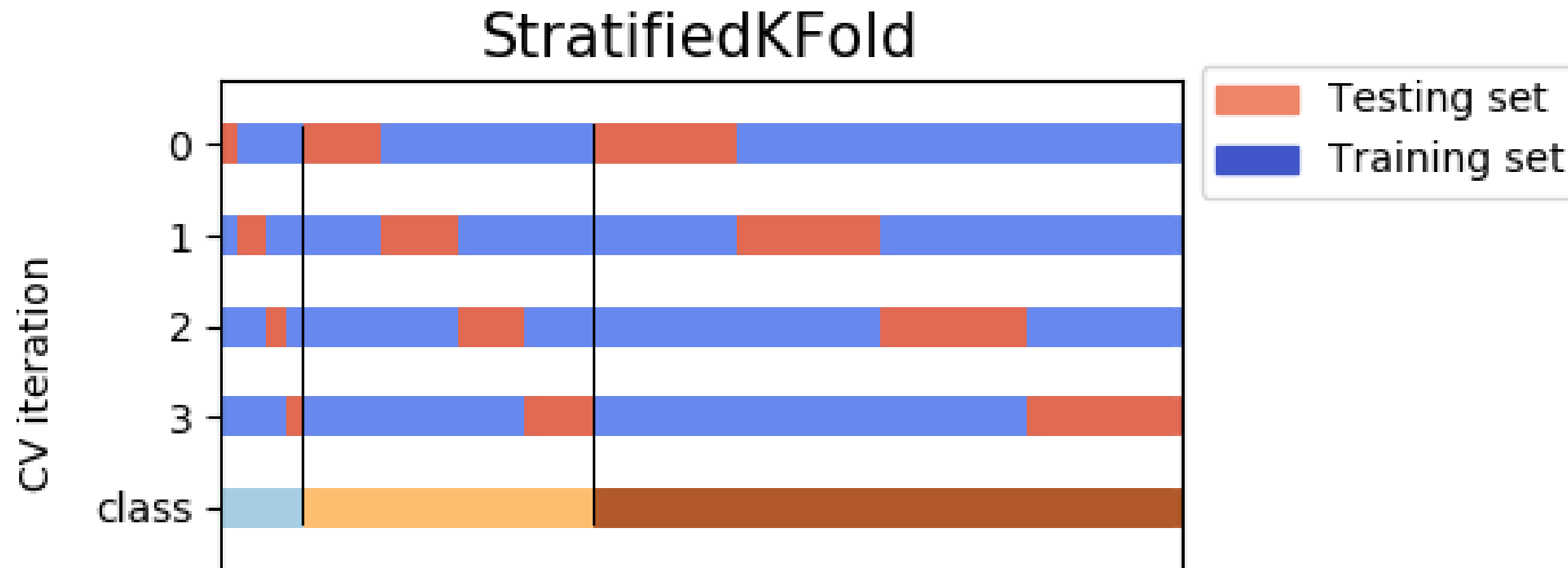
2) for loop에서 split()를 이용해 학습/테스트 데이터 인덱스 추출

3) 반복마다 학습(fit), 예측(predict), 성능(accuracy_score) 반환

Stratified KFold

→ 불균형한 분포도를 가진 레이블 데이터 집합을 위한 KFold 방식

- K-Fold : 데이터를 k개로 나누고 각 폴드를 한번씩 테스트 세트로 사용
- Stratified K-Fold: 각 폴드가 원본 데이터의 클래스비율을 유지하도록 샘플링함
- Regression에서는 제공X, 일반적인 Classification 검증 방식



Stratified KFold

StratifiedKFold객체.split(데이터셋(X값),레이블)

- StratifiedKFold는 y가 없으면 클래스 비율을 유지할 수 없으므로 split() 호출 시 y가 반드시 필요!

```
skfold = StratifiedKFold(n_splits=3)
n_iter=0
cv_accuracy=[]

for train_index, test_index in skfold.split(features, label):
    # split( )으로 반환된 인덱스를 이용해 학습용, 검증용 테스트 데이터 추출
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]
```

```
#학습 및 예측
dt_clf.fit(X_train, y_train)
pred = dt_clf.predict(X_test)
# 반복 시마다 정확도 측정
n_iter += 1
accuracy = np.round(accuracy_score(y_test, pred), 4)
train_size = X_train.shape[0]
test_size = X_test.shape[0]

print('\n#{0} 교차 검증 정확도 :{1}, 학습 데이터 크기: {2}, 검증 데이터 크기: {3}'
      .format(n_iter, accuracy, train_size, test_size))

print('#{0} 검증 세트 인덱스:{1}'.format(n_iter, test_index))
cv_accuracy.append(accuracy)

# 교차 검증별 정확도 및 평균 정확도 계산
print('\n### 교차 검증별 정확도:', np.round(cv_accuracy, 4))
print('## 평균 검증 정확도:', np.round(np.mean(cv_accuracy), 4))
```

```
#1 교차 검증 정확도 :0.98, 학습 데이터 크기: 100, 검증 데이터 크기: 50
#1 검증 세트 인덱스:[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 50
 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 100 101
102 103 104 105 106 107 108 109 110 111 112 113 114 115]

#2 교차 검증 정확도 :0.94, 학습 데이터 크기: 100, 검증 데이터 크기: 50
#2 검증 세트 인덱스:[ 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 67
 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 116 117 118
119 120 121 122 123 124 125 126 127 128 129 130 131 132]

#3 교차 검증 정확도 :0.98, 학습 데이터 크기: 100, 검증 데이터 크기: 50
#3 검증 세트 인덱스:[ 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 83 84
 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 133 134 135
136 137 138 139 140 141 142 143 144 145 146 147 148 149]

## 교차 검증별 정확도: [0.98 0.94 0.98]
## 평균 검증 정확도:, 0.9667
```

cross_val_score()

cross_val_score(**estimator**, **X**, y=None, **scoring**=None, **cv**=None, n_jobs=1, verbose=0, fit_params=None, pre_dispatch= '2 * n_jobs').

- 1) K값 설정
- 2) for loop에서 split()를 이용해 학습/테스트 데이터 인덱스 추출 > cross_val_score()
- 3) 반복마다 학습(fit), 예측(predict), 성능(accuracy_score) 반환

estimator	Classifier – Stratified Kfold 방식 Regressor – Kfold 방식
X	피쳐 데이터 세트
y	레이블 데이터 세트
scoring	예측 성능 평가 지표 디폴트: 해당 estimator의 디폴트 성능 평가 지표
cv	교차검증 폴드 수 디폴트값 : 5
반환값	Scoring에서 지정한 성능 지표 측정값을 배열 타입으로 반환

```
scores = cross_val_score(dt_clf, data, label, scoring='accuracy',cv=3)
print('교차 검증별 정확도:', np.round(scores, 4))
print('평균 검증 정확도:', np.round(np.mean(scores), 4))
```

교차 검증별 정확도: [0.98 0.94 0.98]
평균 검증 정확도: 0.9667

2-04 , 2-05



#2-4 Model Selection Module_GridSearchCV

Keyword: 하이퍼 파라미터

➔ 머신러닝 알고리즘을 구성하는 주요 구성 요소, 파라미터 값을 조정하여 알고리즘의 예측 성능 개선

In 사이킷런 – GridSearchCV API

알고리즘에 사용되는 하이퍼 파라미터를 순차적으로 입력, 편리하게 최적의 파라미터를 도출할 수 있는 방식

Ex) 결정 트리 알고리즘에 적용

```
[1] grid_paramete = {'max_depth': [1, 2, 3], 'min_depth': [2, 3]}
```

#2-4 Model Selection Module_GridSearchCV

예시

If CV = 3회 ⇔ 개별 파라미터 조합마다 3개 폴딩 세트를 3회에 걸쳐 학습/평가
⇔ 평균값으로 성능 측정

Ex) 6개의 파라미터 조합 -> 총 CV 3회 x 6개 파라미터 조합 = 18회 학습 / 평가

GridSearchCV 클래스 생성자로 들어가는 주요 파라미터

estimator	Classifier, regressor, pipeline이 사용됨
param_grid	Key + 리스트 값을 가지는 딕셔너리, estimator의 튜닝을 위해 파라미터명과 사용될 파라미터 값 저장
scoring	예측 성능 측정 평가 방법 지정
cv	교차 검증을 위해 분할되는 학습 / 테스트 세트 개수 지정
refit	Default = True / True로 생성 시 가장 최적의 하이퍼 파라미터 찾고 -> 입력된 estimator 객체를 해당 하이퍼 파라미터로 재학습

#2-4 Model Selection Module_GridSearchCV

예제 / GridSearchCV 이용

결정 트리 알고리즘의 최적화 파라미터 순차 적용하여 붓꽃 데이터 예측 분석

```
[3] from sklearn.datasets import load_iris
    from sklearn.tree import DecisionTreeClassifier
    from sklearn.model_selection import GridSearchCV
    # Import train_test_split
    from sklearn.model_selection import train_test_split

    # 데이터를 로딩하고 학습 데이터와 테스트 데이터 분리
    iris_data = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target, test_size=0.2, random_state=121)
    dtree = DecisionTreeClassifier()

    # ## 파라미터를 딕셔너리 형태로 설정
    parameters = {'max_depth': [1, 2, 3], 'min_samples_split': [2, 3]}
```

#2-4 Model Selection Module_GridSearchCV

예제 / 주요 칼럼 발췌, 동작 구조 파악

```
import pandas as pd
# param_grid의 하이퍼 파라미터를 3개의 train, test set fold로 나누어 테스트 수행 설정 .
# ## refit=True가 default임. True이면 가장 좋은 파라미터 설정으로 재학습시킴.

grid_dtree = GridSearchCV(dtree, param_grid=parameters, cv=3, refit=True)

# 붓꽃 학습 데이터로 param_grid의 하이퍼 파라미터를 순차적으로 학습/평가 .
grid_dtree.fit(X_train, y_train)

# GridSearchCV 결과를 추출해 DataFrame으로 변환
scores_df = pd.DataFrame(grid_dtree.cv_results_)
scores_df[['params', 'mean_test_score', 'rank_test_score', 'split0_test_score',
            'split1_test_score', 'split2_test_score']]
```

	params	mean_test_score	rank_test_score	split0_test_score	split1_test_score	split2_test_score
0	{'max_depth': 1, 'min_samples_split': 2}	0.700000	5	0.700	0.7	
1	{'max_depth': 1, 'min_samples_split': 3}	0.700000	5	0.700	0.7	
2	{'max_depth': 2, 'min_samples_split': 2}	0.958333	3	0.925	1.0	
3	{'max_depth': 2, 'min_samples_split': 3}	0.958333	3	0.925	1.0	
4	{'max_depth': 3, 'min_samples_split': 2}	0.975000	1	0.975	1.0	
5	{'max_depth': 3, 'min_samples_split': 3}	0.975000	1	0.975	1.0	

주요 칼럼별 의미 정리

- params 칼럼: 수행할 때마다 적용된 개별 하이퍼 파라미터값
- rank_test_score: 하이퍼 파라미터별로 성능이 좋은 score 순위 (1이 최고, 이때 파라미터 = 최적)
- mean_test_score: 개별 하이퍼 파라미터별로 CV 폴딩 테스트 세트에 대해 총 수행한 평가 평균값

#2-4 Model Selection Module_GridSearchCV

예제 / 최적 하이퍼 파라미터 값과 그때의 정확도

```
[7] print('GridSearchCV 최적 파라미터:', grid_dt.tree.best_params_)  
    print('GridSearchCV 최고 정확도:{0:.4f}'.format(grid_dt.tree.best_score_))
```

```
GridSearchCV 최적 파라미터: {'max_depth': 3, 'min_samples_split': 2}  
GridSearchCV 최고 정확도:0.9750
```

#2-4 Model Selection Module_GridSearchCV

예제 / 테스트 데이터 세트에 대해 예측, 성능 평가

```
✓ 0초 [9] # GridSearchCV의 ref it 으로 이미 학습된 estimator 반환

estimator = grid_dtrees.best_estimator_

# GridSearchCV의 best_estimator_는 이미 최적 학습이 됐으므로 별도 학습이 필요 없음
from sklearn.metrics import accuracy_score # Import accuracy_score from sklearn.metrics
pred = estimator.predict(X_test)
print('테스트 데이터 세트 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

→ 테스트 데이터 세트 정확도: 0.9667

#2-5 데이터 전처리

#1 데이터 인코딩

#2 피처 스케일링 / 정규화

#3 StandardScaler

#4 MinMaxScaler

#5 스케일링 시 유의점

#2-5 데이터 전처리_preview

데이터 전처리(Data Preprocessing)

중요성 – ML 알고리즘은 데이터 기반, 따라서 입력 데이터에 따라 아웃풋도 달라짐!
사이킷런 – 데이터에 대해 미리 처리해야 하는 기본 사항 有

기본 사항 1) 결손값 (NaN, Null값)	기본 사항 2) 문자열 값
a) 피쳐 값 중 Null 값이 적을 경우 – 피쳐의 평균값으로 대체 b) Null 값이 대부분 – 해당 피쳐 드롭 c) ** Null 값이 일정 수준 이상 – 대체 값 선정 ↳ 해당 피쳐의 중요도가 높으며, 평균값으로 대체 시 예측 왜곡이 심하다면	In 사이킷런 ML 알고리즘, 문자열 값을 입력값으로 허용 X ➔ 문자열 값을 인코딩하여 숫자 형으로 변환해야 함! 문자열 피쳐 -> 카테고리형 피쳐, 텍스트형 피쳐 카테고리형 피쳐 -> 코드 값 텍스트형 피쳐 -> 피쳐 벡터화 기법으로 벡터화 or 삭제 Ex) 주민번호, 단순 문자열 아이디 (단순 식별용)

#2-5 데이터 전처리_데이터 인코딩

레이블 인코딩(Label encoding) – 카테고리 피처를 코드형 숫자 값으로 변환

** LabelEncoder 클래스로 구현 / LabelEncoder를 객체로 생성, fit()과 transform() 호출

```
[10] from sklearn.preprocessing import LabelEncoder

items = ['TV', '냉장고', '전자레인지', '컴퓨터', '선풍기', '선풍기', '믹서', '믹서']

# LabelEncoder를 객체로 생성한 후, fit( )과 transform( )으로 레이블 인코딩 수행.
encoder = LabelEncoder()
encoder.fit(items)
labels = encoder.transform(items)
print('인코딩 변환값:', labels)
```

인코딩 변환값: [0 1 4 5 3 3 2 2]

```
[11] print('인코딩 클래스:', encoder.classes_)
```

인코딩 클래스: ['TV' '냉장고' '믹서' '선풍기' '전자레인지' '컴퓨터']

```
[12] print('디코딩 원본값:', encoder.inverse_transform([4, 5, 2, 0, 1, 1, 3, 3]))
```

디코딩 원본값: ['전자레인지' '컴퓨터' '믹서' 'TV' '냉장고' '냉장고' '선풍기' '선풍기']

#2-5 데이터 전처리-데이터 인코딩

레이블 인코딩(Label encoding) – 카테고리 피처를 코드형 숫자 값으로 변환

원본 데이터		→		상품 분류를 레이블 인코딩한 데이터	
상품 분류	가격			상품 분류	가격
TV	1,000,000			0	1,000,000
냉장고	1,500,000			1	1,500,000
전자레인지	200,000			4	200,000
컴퓨터	800,000			5	800,000
선풍기	100,000			3	100,000
선풍기	100,000			3	100,000
믹서	50,000			2	50,000
믹서	50,000			2	50,000

Feature

- 간단하게 문자열 값을 숫자형 카테고리 값으로 변환 가능
 - 일괄적인 숫자 값으로 변환 시, 몇몇 ML 알고리즘에서 예측 성능 저하 발생
 - ↳ 숫자 값의 경우 크고 작음의 특성 작용, 특정 ML 알고리즘에서 큰 수에 가중치 부여
- But, 문자열 값의 숫자 변환 값은 단순 코드, not 순서 or 중요도 역할
- => 선형 회귀 등의 알고리즘에 적용 X / 트리 계열 ML 알고리즘은 반영 X 사용 가능

#2-5 데이터 전처리_데이터 인코딩

원-핫 인코딩(One-Hot Encoding)

피쳐 값의 유형에 따라 새로운 피쳐 추가, 고유 값에 해당하는 칼럼에만 1을 표시 – 나머지 칼럼에는 0 표시

행 형태의 피쳐 고유 값을 열 형태로 차원 변환, 고유 값 해당 칼럼에만 1 표시, 나머지 칼럼에는 0 표시

원본 데이터		원-핫 인코딩					
상품 분류		상품분류_ TV	상품분류_ 냉장고	상품분류_ 믹서	상품분류_ 선풍기	상품분류_ 전자레인지	상품분류_ 컴퓨터
TV		1	0	0	0	0	0
냉장고	→	0	1	0	0	0	0
전자레인지		0	0	0	0	1	0
컴퓨터	→	0	0	0	0	0	1
선풍기		0	0	0	1	0	0
선풍기		0	0	0	1	0	0
믹서		0	0	1	0	0	0
믹서		0	0	1	0	0	0

#2-5 데이터 전처리_데이터 인코딩

원-핫 인코딩(One-Hot Encoding)

In 사이킷런, OneHotEncoder 클래스로 변환

Feature

- 입력값으로 2차원 데이터 필요
- OnehotEncoder 이용한 변환 값이 희소 행렬(Sparse Matrix) 형태 -> toarray() 메서드 이용, 밀집 행렬(Dense Matrix)로 변환

```
[13] from sklearn.preprocessing import OneHotEncoder
import numpy as np
items = ['TV', '냉장고', '전자레인지', '컴퓨터', '선불기', '선불기', '믹서', '믹서']

# 2차원 ndarray로 변환합니다.
items = np.array(items).reshape(-1, 1)

# 원-핫 인코딩을 적용합니다.
oh_encoder = OneHotEncoder()
oh_encoder.fit(items)
oh_labels = oh_encoder.transform(items)

# OneHotEncoder로 변환한 결과는 희소행렬이므로 toarray()를 이용해 밀집 행렬로 변환.
print('원-핫 인코딩 데이터')
print(oh_labels.toarray())
print('원-핫 인코딩 데이터 차원')
print(oh_labels.shape)
```

원-핫 인코딩 데이터

1.	0.	0.	0.	0.	0.
0.	1.	0.	0.	0.	0.
0.	0.	0.	0.	1.	0.
0.	0.	0.	0.	0.	1.
0.	0.	0.	1.	0.	0.
0.	0.	0.	1.	0.	0.
0.	0.	1.	0.	0.	0.
0.	0.	1.	0.	0.	0.

원-핫 인코딩 데이터 차원
(8, 6)

#2-5 데이터 전처리_데이터 인코딩

원-핫 인코딩(One-Hot Encoding)

판다스 get_dummies() API – 원-핫 인코딩 지원

- 문자열 카테고리 값을 숫자 형으로 변환할 필요 X, 바로 변환 가능

✓
0초

[14]

```
import pandas as pd
df = pd.DataFrame({'item': ['TV', '냉장고', '전자레인지', '컴퓨터', '선풍기', '선풍기', '믹서', '믹서']})
pd.get_dummies(df)
```

	item_TV	item_냉장고	item_믹서	item_선풍기	item_전자레인지	item_컴퓨터
0	True	False	False	False	False	False
1	False	True	False	False	False	False
2	False	False	False	False	True	False
3	False	False	False	False	False	True
4	False	False	False	True	False	False
5	False	False	False	True	False	False
6	False	False	True	False	False	False
7	False	False	True	False	False	False

#2-5 데이터 전처리 - 피쳐 스케일링 / 정규화

피쳐 스케일링(feature scaling)

서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업

- 표준화(Standardization)
- 데이터의 피쳐 각각이 평균이 0, 분산이 1인 가우시안 정규 분포를 가진 값으로 변환

$$x_{i_new} = \frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$$

- 정규화(Normalization)
- 서로 다른 피쳐의 크기를 통일하기 위해 크기를 변환해주는 개념, 개별 데이터의 크기를 모두 똑같은 단위로 변경

$$x_{i_new} = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

#2-5 데이터 전처리_피쳐 스케일링 / 정규화

피쳐 스케일링(feature scaling)

서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업

- 사이킷런의 전처리 Normalize 모듈
- 선형 대수에서의 정규화 개념 적용, 개별 벡터의 크기를 맞추기 위해 변환
-> 개별 벡터를 모든 피쳐 벡터의 크기로 나눠 줌

$$x_{i_new} = \frac{x_i}{\sqrt{x_i^2 + y_i^2 + z_i^2}}$$

#2-5 데이터 전처리_StandardScaler

StandardScaler

- 표준화 지원 클래스
- 개별 피쳐 평균이 0, 분산이 1인 값으로 변환(가우시안 정규 분포를 가질 수 있도록 데이터 변환)
- 중요성
 - RBF 커널을 이용하는 서포트 벡터 머신, 선형 회귀, 로지스틱 회귀 = 데이터가 가우시안 분포를 가지고 있다고 가정하고 구현
 - 사전에 표준화 적용이 예측 성능 향상에 중요한 요소

```
[15] from sklearn.datasets import load_iris
import pandas as pd

# 붓꽃 데이터 세트를 로딩하고 DataFrame으로 변환합니다.
iris = load_iris()
iris_data = iris.data
iris_df = pd.DataFrame(data=iris_data, columns=iris.feature_names)
print('feature 들의 평균 값')
print(iris_df.mean())
print('\nfeature 들의 분산 값')
print(iris_df.var())
```

```
feature 들의 평균 값
sepal length (cm)    5.843333
sepal width (cm)     3.057333
petal length (cm)    3.758000
petal width (cm)     1.199333
dtype: float64

feature 들의 분산 값
sepal length (cm)    0.685694
sepal width (cm)     0.189979
petal length (cm)    3.116278
petal width (cm)     0.581006
dtype: float64
```

#2-5 데이터 전처리_StandardScaler

StandardScaler

```
✓ [17] from sklearn.preprocessing import StandardScaler
0초

# StandardScaler객체 생성
scaler = StandardScaler()
# StandardScaler로 데이터 세트 변환. fit( )과 transform( ) 호출.
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

# transform( ) 시 스케일 변환된 데이터 세트가 NumPy ndarray로 반환돼 이를 DataFrame으로 변환

iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)
print('feature 들의 평균 값')
print(iris_df_scaled.mean())
print('feature 들의 분산 값')
print(iris_df_scaled.var())
```

↔ feature 들의 평균 값

sepal length (cm)	-1.690315e-15
sepal width (cm)	-1.842970e-15
petal length (cm)	-1.698641e-15
petal width (cm)	-1.409243e-15

dtype: float64

feature 들의 분산 값

sepal length (cm)	1.006711
sepal width (cm)	1.006711
petal length (cm)	1.006711
petal width (cm)	1.006711

dtype: float64

#2-5 데이터 전처리_MinMaxScaler

MinMaxScaler

- 데이터값을 0과 1 사이의 범위 값으로 변환
- 음수 값이 있을 시 -1에서 1값으로 변환

```
[18] from sklearn.preprocessing import MinMaxScaler

# MinMaxScaler객체 생성
scaler = MinMaxScaler()
# MinMaxScaler로 데이터 세트 변환. fit()과 transform() 호출.
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

# transform() 시 스케일 변환된 데이터 세트가 NumPy ndarray로 반환돼 이를 DataFrame으로 변환
iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)
print('feature 들의 최솟값')
print(iris_df_scaled.min())
print('feature들의 최댓값')
print(iris_df_scaled.max())
```

```
feature 들의 최솟값
sepal length (cm)    0.0
sepal width (cm)     0.0
petal length (cm)    0.0
petal width (cm)     0.0
dtype: float64

feature들의 최댓값
sepal length (cm)    1.0
sepal width (cm)     1.0
petal length (cm)    1.0
petal width (cm)     1.0
dtype: float64
```


#2-5 데이터 전처리_스케일링 시의 유의점

Scaler 객체 이용 데이터 스케일링 변환 시 사용 메서드

- fit(): 데이터 변환을 위한 기준 정보 설정
- transform(): 해당 설정 정보 이용 데이터 변환
- fit_transform(): fit(), transform() 한 번에 적용 기능 수행

주의점

- 학습 데이터 세트 & 테스트 데이터 세트에 fit()과 transform() 적용 시 주의
- Scaler 객체를 이용해 학습 데이터 세트로 메서드 적용
-> 테스트 데이터 세트로 다시 fit() 수행 X / 학습 데이터 세트 fit() 수행 결과 이용해 transform() 변환 적용
- 학습 데이터로 fit()이 적용된 스케일링 기준 정보 그대로 테스트 데이터에 적용
- 테스트 데이터로 다시 새로운 스케일링 기준 정보 만들 시, 학습 데이터와 테스트 데이터의 스케일링 기준 정보 달라짐
↳ 올바른 예측 결과 도출 X
- fit_transform()도 테스트 데이터에서 사용 X
- 따라서... 학습/테스트 데이터 세트로 분리 이전에 전체 데이터 세트에서 스케일링 적용 후 분리

#2-5 데이터 전처리_스케일링 시의 유의점

✓
0초

```
[19] from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 학습 데이터는 0부터 10까지, 테스트 데이터는 0부터 5까지 값을 가지는 데이터 세트로 생성
# Scaler 클래스의 fit(), transform은 2차원 이상 데이터만 가능하므로 reshaped(1, 1)로 차원 변경

train_array = np.arange(0, 11).reshape(-1, 1)
test_array = np.arange(0, 6).reshape(-1, 1)
```

✓
0초

```
[21] # MinMaxScaler 객체에 별도의 feature_range 파라미터 값을 지정하지 않으면 0~1 값으로 변환
scaler = MinMaxScaler()

# fit()하게 되면 train_array 데이터의 최솟값이 0, 최댓값이 10으로 설정.
scaler.fit(train_array)

# 1/10 scale로 train_array 데이터 변환함. 원본 10-> 1로 변환됨.
train_scaled = scaler.transform(train_array)

print('원본 train_array 데이터:', np.round(train_array.reshape(-1), 2))
print('Scale된 train_array 데이터:', np.round(train_scaled.reshape(-1), 2))
```

⇒ 원본 train_array 데이터: [0 1 2 3 4 5 6 7 8 9 10]
Scale된 train_array 데이터: [0. 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.]

#2-5 데이터 전처리_스케일링 시의 유의점

✓ 0초 [23] # MinMaxScaler에 test_array를 fit()하게 되면 원본 데이터의 최솟값이 0, 최댓값이 5로 설정됨
scaler.fit(test_array)

1/5 scale로 test_array 데이터 변환함. 원본 5→ 1로 변환.
test_scaled = scaler.transform(test_array)

test_array의 scale 변환 출력.

print('원본 test_array 데이터:', np.round(test_array.reshape(-1), 2))

print('Scale된 test_array 데이터:', np.round(test_scaled.reshape(-1), 2))

⇒ 원본 test_array 데이터: [0 1 2 3 4 5]
Scale된 test_array 데이터: [0. 0.2 0.4 0.6 0.8 1.]

✓ 0초 [24] scaler = MinMaxScaler()
scaler.fit(train_array)
train_scaled = scaler.transform(train_array)
print('원본 train_array 데이터:', np.round(train_array.reshape(-1), 2))
print('Scale된 train_array 데이터:', np.round(train_scaled.reshape(-1), 2))

test_array에 Scale 변환을 할 때는 반드시 fit()을 호출하지 않고 transform()만으로 변환해야 함.

test_scaled = scaler.transform(test_array)

print('원본 test_array 데이터:', np.round(test_array.reshape(-1), 2))

print('Scale된 test_array 데이터:', np.round(test_scaled.reshape(-1), 2))

⇒ 원본 train_array 데이터: [0 1 2 3 4 5 6 7 8 9 10]
Scale된 train_array 데이터: [0. 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.]

원본 test_array 데이터: [0 1 2 3 4 5]

Scale된 test_array 데이터: [0. 0.1 0.2 0.3 0.4 0.5]

3장 . 평가



#3.1 정확도 (Accuracy)

정확도란?

: 실제 데이터와 예측 데이터가 얼마나 같은지 판단하는 지표

$$\text{정확도(Accuracy)} = \frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$$

실제 데이터	1	1	1	1	1	1	1	1	1	0
예측 데이터	1	1	1	1	1	1	1	1	1	1

정확도 90%

#3.2 오차 행렬 (confusion matrix)

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

```
from sklearn.metrics import confusion_matrix
```

```
confusion_matrix(y_test, fakepred)
```

【Output】

```
array([[405,  0],  
       [ 45,  0]], dtype=int64)
```


#3.2 오차 행렬 (confusion matrix)

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, fakepred)
```

【Output】

```
array([[405,  0],
       [ 45,  0]], dtype=int64)
```

정확도 = 예측이 정확한 데이터 / 전체 데이터
= (TN+TP)/(TN+FP+FN+TP)

#3.3 정밀도와 재현율

정밀도 = $TP / (FP + TP)$ = 양성으로 예측한 것 중 진짜 양성인 비율

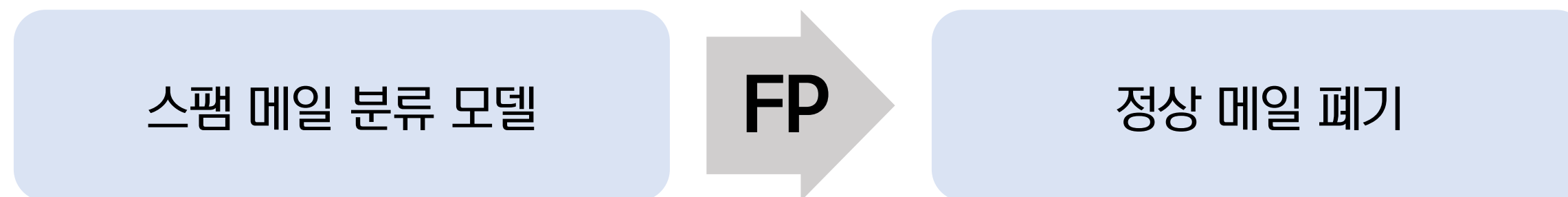
재현율 = $TP / (FN + TP)$ = 실제 양성인 것 중 양성으로 예측된 비율

```
precision = precision_score(y_test, pred)
recall = recall_score(y_test, pred)
```

#3.3 정밀도와 재현율의 트레이드 오프

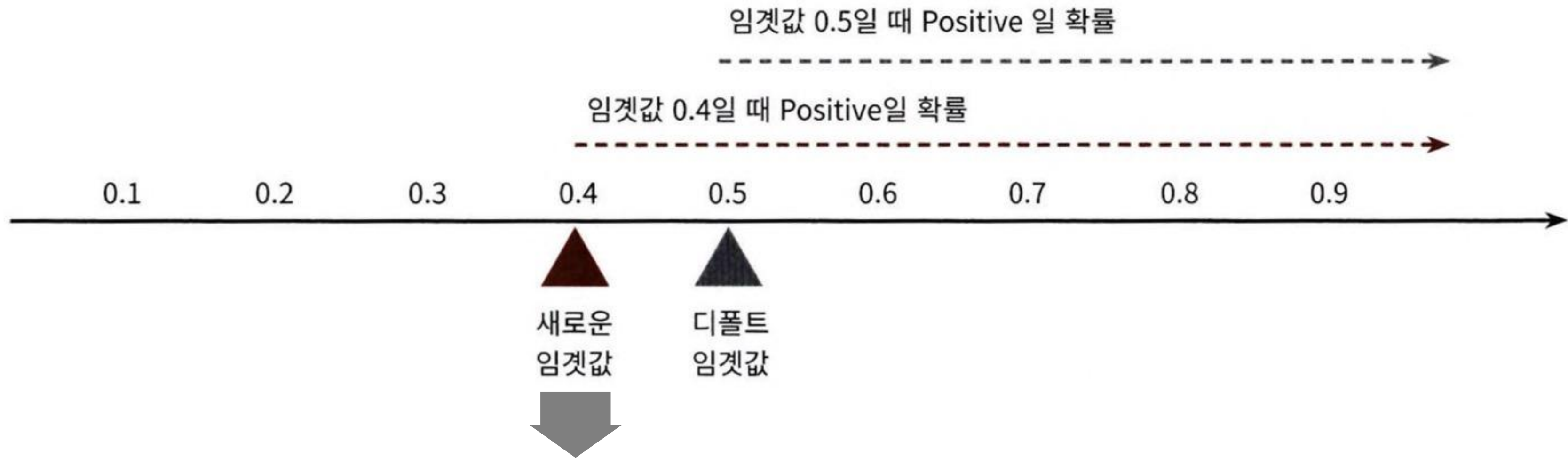


$$\text{재현율} = TP / (FN + TP)$$



$$\text{정밀도} = TP / (FP + TP)$$

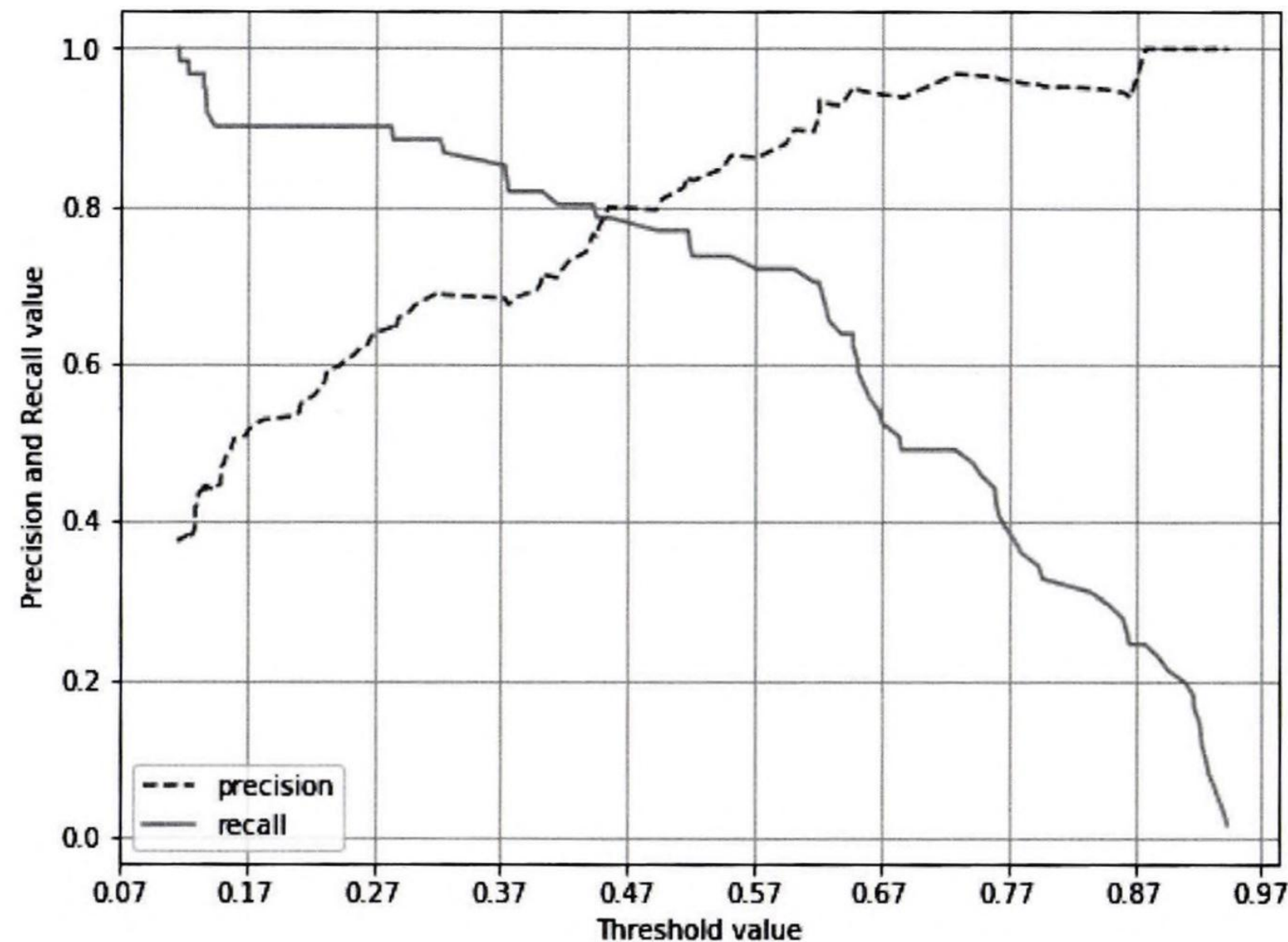
#3.3 정밀도와 재현율의 트레이드 오프



- FN 감소
- 재현율 증가
$$= TP / (FN + TP)$$

#3.3 정밀도와 재현율의 트레이드 오프

```
# 실제값 데이터 세트와 레이블 값이 1일 때의 예측 확률을 precision_recall_curve 인자로 입력
precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_class1 )
print('반환된 분류 결정 임계값 배열의 Shape:', thresholds.shape)
```



#3.3 정밀도와 재현율의 맹점

정밀도 100% 만들기

실제 데이터	1	0	1	0	0	...	0	1	1	0
예측 데이터	1	0	0	0	0	...	0	0	0	0

$$\text{정밀도} = \text{TP} / (\text{FP} + \text{TP}) = 1 / (0 + 1) = \mathbf{1}$$

재현율 100% 만들기

실제 데이터	1	0	1	0	0	...	0	1	1	0
예측 데이터	1	1	1	1	1	...	1	1	1	1

$$\text{재현율} = \text{TP} / (\text{FN} + \text{TP}) = N / (0 + N) = \mathbf{1}$$

#3.4 F1 Score

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

Precision = 0.9 , Recall = 0.1

→ $F1 = 2 * (0.09) / (1) = \mathbf{0.18}$

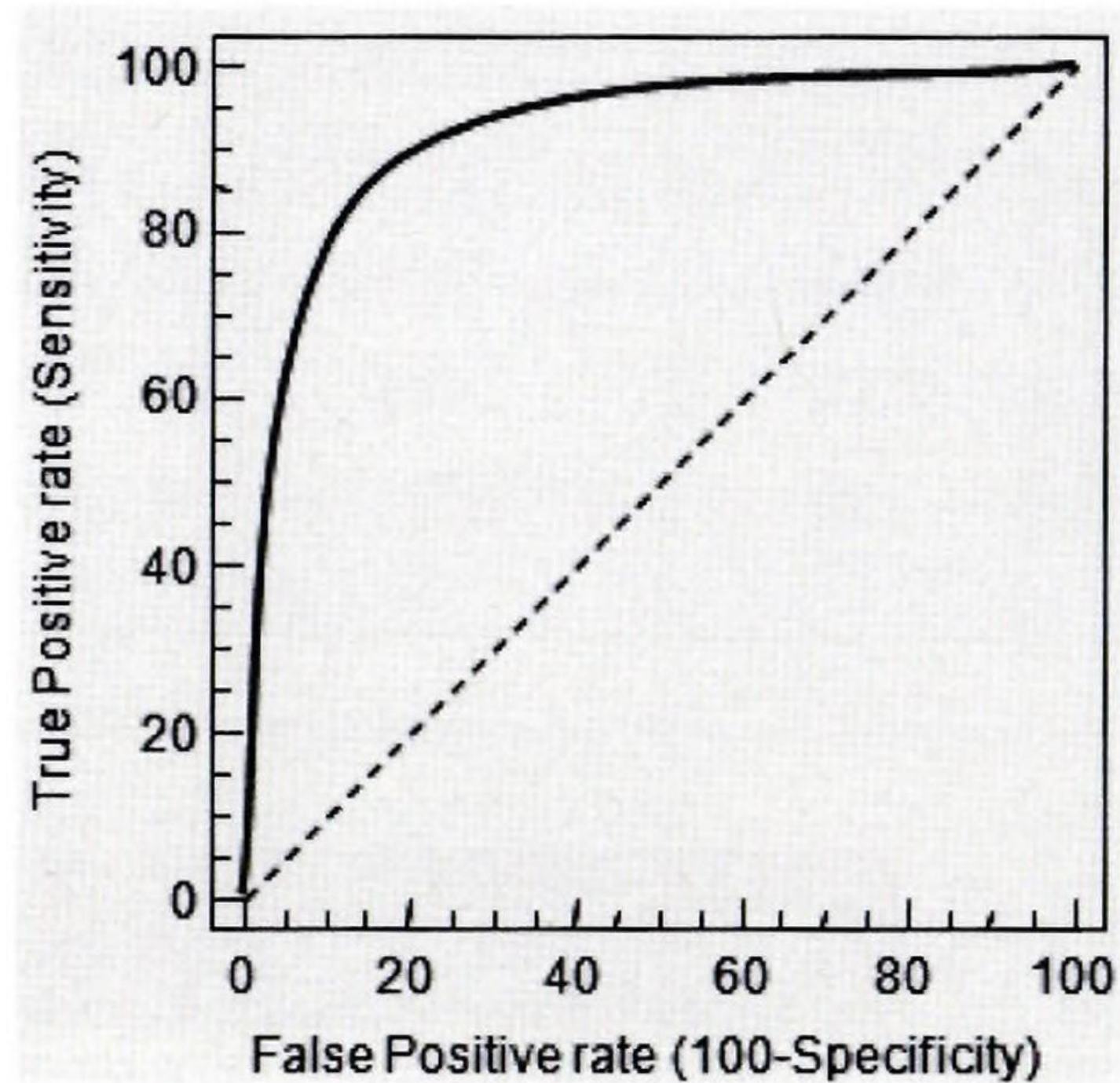
Precision = 0.5 , Recall = 0.5

→ $F1 = 2 * (0.25) / 1 = \mathbf{0.5}$

#3.5 ROC Curve, AUC

- $\text{TPR}(\text{True Positive Rate}) = \text{TP}/(\text{FN}+\text{TP}) = \text{재현율 (민감도)}$
- $\text{TNR}(\text{True Negative Rate}) = \text{TN}/(\text{FP}+\text{TN}) = \text{특이성}$
- $\text{FPR}(\text{False Positive Rate}) = \text{FP}/(\text{FP}+\text{TN}) = 1 - \text{TNR}$

#3.5 ROC Curve, AUC



〈 ROC 곡선 예시 〉

THANK YOU

